Summary

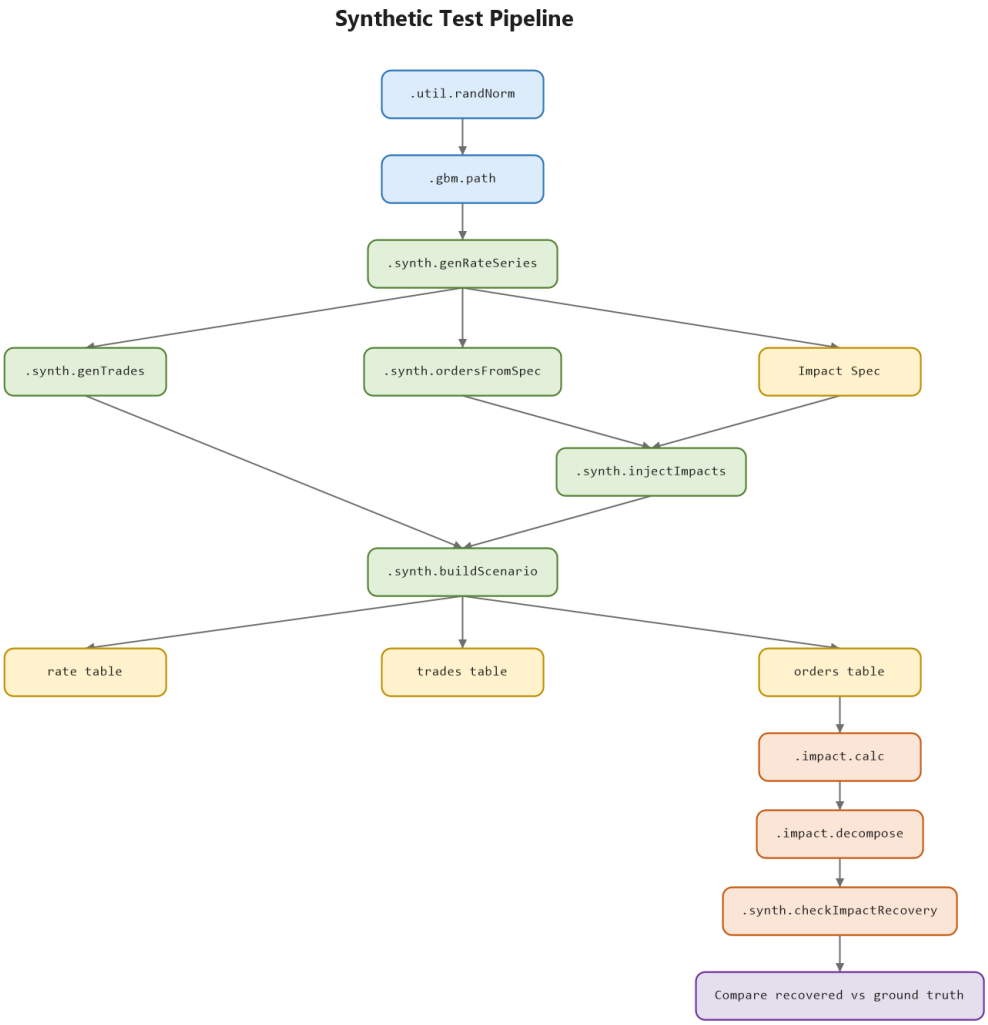
Markout and Market Impact are often viewed hand-in-hand and are central to post-trade analytics. In the context of this article and the code we categorise them in the following way:

- Markout asks: after a client's trade executes, did the market keep moving in the direction they traded? It's a measure of who benefited from the timing of a fill — useful for client tiering, venue comparison, and understanding the real cost of internalising flow.
- Market impact asks something narrower and more mechanical: when your own order hits the market, how much does it move the price, and does that move stick or fade? It's a measure of your own execution footprint — useful for tuning algo aggression, sizing orders, and understanding the true cost of demanding liquidity right now versus waiting.

Repo

- The code discussed in this article can be found at: https://github.com/SpencerFX/q-link
- To load the functions, test data, and run the pipeline from within the q-Link directory:

```
q ./scripts/initMarkout.q
```



Use grids to vectorise time-series analysis

Rather than executing a separate join for each temporal offset, the offset grid is constructed once and subsequently crossed with the set of events. This procedure yields a single row for every combination of event and offset, together with the timestamp required for the reference price lookup.

The resulting table may then be sorted and passed through a single aj operation, which performs all reference price lookups simultaneously rather than requiring one per offset.

A further detail specific to q merits attention within .util.explode. The crossed table is first assigned to the variable crossed prior to the update statement, since timeCol holds the name of the time column rather than the column's contents. Within the update expression, q requires the actual column values; accordingly, the column is accessed by evaluating crossed[timeCol].

```
.util.buildGrid:{[posOffsets]
  pos:asc distinct posOffsets except 0f;
  (reverse neg pos),0f,pos
 };
.util.explode:{[rows;timeCol;gridNS]
  crossed:rows cross ([]offset:gridNS);
  update targetTime:crossed[timeCol]+offset from crossed
 };
```

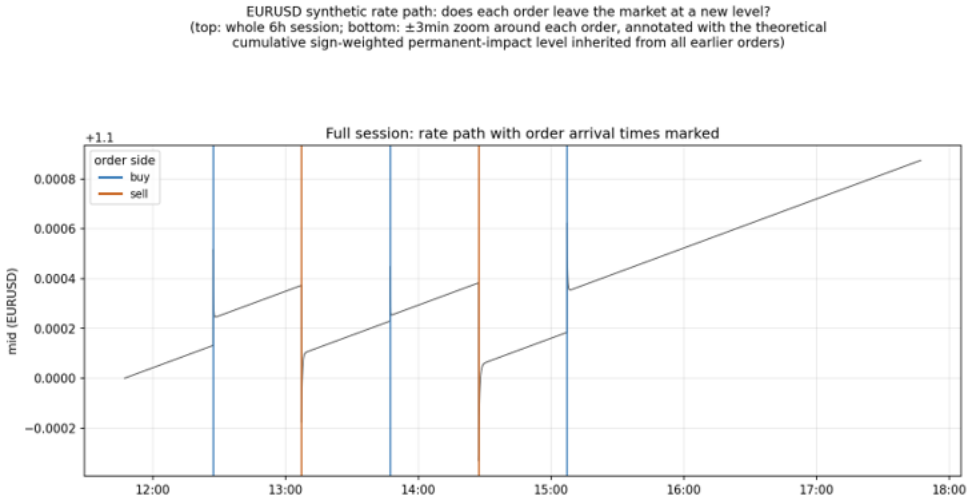
On Data

To validate the implementation, the project includes a synthetic data generator that simulates an FX market using Geometric Brownian Motion (GBM). From this baseline market it generates synthetic trades and orders, then injects a known market-impact profile with configurable temporary impact, permanent impact, and exponential decay. Because these parameters are specified in advance, the ground truth is known exactly, allowing .impact.decompose to be quantitatively evaluated against the injected values rather than relying solely on visual inspection of impact curves.

Note: GBM is not intended to be a realistic model of FX microstructure. Its purpose here is to provide a controlled stochastic baseline with known drift and volatility, against which the markout and impact calculations can be tested. The important property is that the baseline can be generated independently of the injected execution footprint.

```
.synth.buildScenario:{[]
  sym:`EURUSD;
  start:.z.p - 0D06:00:00;      / 6-hour synthetic session
  dtSecs:0.5;                   / tick every 500ms
  / baseline path: mild drift so markout has a non-zero, checkable
  / expected value (expected markout at offset D ~ mu*D)
  base:.synth.genRateSeries[sym;start;6*60*60;dtSecs;1.1000;5e-8;2e-5];
  / plan five well-separated impact events, alternating buy/sell,
  orderTimes:start+`timespan$1e9*40*60*1+til 5;
  spec:([] orderTime:orderTimes; sym:5#sym;
    dirSign:1 -1 1 -1 1f;
    tempBps:3.5 5.0 2.0 6.5 4.0;
    permBps:1.0 2.5 0.2 3.0 1.5;
    halfLifeSecs:8 15 5 20 10f);
  orders:.synth.ordersFromSpec[spec;base];
  rate:.synth.injectImpacts[base;spec];
  trades:.synth.genTrades[sym;2000;base;0.3];
  `rate`trades`orders`groundTruth!(rate;trades;orders;spec) };
```

In the synthetic environment, the injected impact is known by construction, so the fitted curve can be compared directly against ground truth. In live data, however, the observed post-order price movement is not automatically causal: it combines the order's footprint with contemporaneous market movement and other sources of price discovery. The parametric fit therefore provides an impact estimate, not proof of causality. A production implementation would typically require a benchmark or control methodology to separate market movement from execution impact.



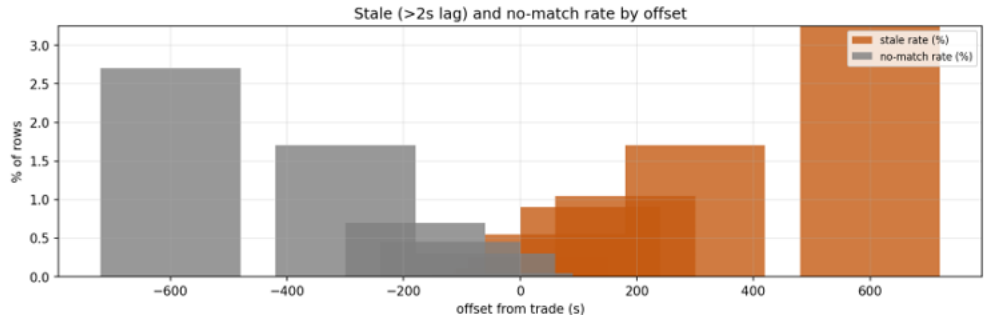
Differences in Approach

For markout we use a wide symmetric grid before and after the trade, out to ten minutes to capture both fast microstructure noise and slower drift:

```
.markout.gridSecs:.util.buildGrid[(0.1*1+til 10),(2+til 14),30 60 120 180 300 600f]
```

Because a single trade's markout is mostly noise, the figure that matters is the notional-weighted aggregate across many trades:

```
select markoutBps:1e4*wavg[notional;markoutVal] by sym,grids from t where not stale
```



Market impact uses a tighter forward-looking window, the seconds immediately after an order, since that is where footprint is actually visible before broader market moves dilute the signal:

```
.impact.gridSecs:.util.buildGrid[(0.5*1+til 20),(11+til 50)];
.impact.gridSecs:.impact.gridSecs where .impact.gridSecs>=-10;
```

We also apply a sign convention relative to the order's own side, so a buy that pushes price up and a sell that pushes price down both register as positive impact rather than cancelling each other out in an aggregate:

```
update impact:dirSign*rawMove from res    / dirSign: +1 buy, -1 sell
```

Temporary vs. Permanent

This is where market impact earns its own analysis, not just a renamed markout. Price impact typically has two components:

- A temporary piece — the price concession paid for demanding liquidity right now, which decays as the market absorbs the trade.
- A permanent piece — the market updating its belief about fair value based on the information the order revealed.

The initial implementation estimated the split by reading two numbers off two arbitrary windows: max impact in an early "peak" window, avg impact in a later "settled" window. This may need to made more robust as the recovered numbers depend on exactly where those thresholds are drawn.

An order with a genuinely slow decay can have its permanent impact overestimated simply because the settled window closes before the curve has actually settled. The more rigorous version treats decay as a real parametric model:

Impact Model

$$I(t) = P + (T - P)e^{\left\{-\frac{t}{\tau}\right\}}$$

and fits temp, perm, and tau jointly. The key insight that makes this tractable in q with no general nonlinear solver is that for any *fixed* tau, the model is linear in (temp, perm). That reduces the problem to a 1-D search over tau alone, with the other two parameters solved exactly in closed form at every candidate via 2×2 normal equations:

```
.impact.fitForTau:{[tau;t;y]
  x1:exp neg t%tau;                 / weight on temp
  x2:1-x1;                          / weight on perm
  a11:sum x1*x1; a12:sum x1*x2; a22:sum x2*x2;
  c1:sum x1*y;   c2:sum x2*y;
  det:a11*a22-a12*a12;
  temp:(c1*a22-c2*a12)%det;
  perm:(a11*c2-a12*c1)%det;
  resid:y-((temp*x1)+perm*x2);
  `temp`perm`tau`sse!(temp;perm;tau;sum resid*resid)
 };
```

A coarse, log-spaced scan over candidate tau values is used to bracket the minimum, following which a golden-section search is employed to refine the estimate. The result consists of three calibrated parameters, rather than two threshold-dependent readings, together with an actual goodness-of-fit measure for each order (r2, rmseBps). Consequently, a poor recovery and a poor fit are no longer indistinguishable from one another. The implied half-life, computed as tau multiplied by ln2, is reported directly in seconds and is therefore directly comparable to the injected ground truth without requiring further conversion.

The fit is restricted to observations for which offsetSec is greater than or equal to zero, since the model describes behavior following an order; the inclusion of pre-order baseline points would bias the temporary and permanent estimates toward whatever value those points happen to average.

Why should the two quantities be separated at all? The reason is that temporary and permanent impact imply opposite responses. High temporary impact indicates that one is paying too much for immediacy, suggesting that it may be preferable to slow down and work the order more gradually.

Temporary vs. Permanent Decomposition

$$I(t) = Te^{\left\{-\frac{t}{\tau}\right\}} + P\left(1 - e^{\left\{-\frac{t}{\tau}\right\}}\right)$$

Elevated permanent impact indicates that the market has genuinely revalued the asset in response to the firm's own order flow. This constitutes a more fundamental problem than one of mere execution style, and it is not one that can be remedied simply by trading at a slower pace. Such elevated permanent impact is observed more frequently in less liquid markets, or in circumstances where the firm's participation rate in a particular asset is comparatively high.

Interpreting the Results

After running the init function the results of the model are available. The impact table exposes the following metrics for comparison:

```
/ Ground Truth vs. Estimated Market Impact
/ ----------------------------------------
/ tempBps             - Temporary impact injected into the synthetic market (ground truth)
/ permBps             - Permanent impact injected into the synthetic market (ground truth)
/ temporaryImpactBps  - Temporary impact estimated by: .impact.decompose
/ permanentImpactBps  - Permanent impact estimated by: .impact.decompose
/ halfLifeSecs        - Half-life recovered by the fitted decay curve (tau * ln2)
/ r2                  - Fraction of variance in the impact curve explained by the fitted model
/ rmseBps             - Root mean squared error of the fit, in basis points
```

r2 and rmseBps are important additions over a simple window-reading approach: a recovered estimate that looks plausible can still come from a poorly fitted curve, and these two numbers make that visible rather than hiding it.

Decay / Half-life

$$t_{\left\{\frac{1}{2}\right\}} = \tau \backslash ln2$$

Model Quality via r2

$$R^2 = 1 - \backslash frac\left\{\sum_{i(y_i - \widehat{\ } y_i)}^{2}\right\}\left\{\sum_{i(y_i - \bar{\ } y)}^{2}\right\}$$

For markout, the following schema is produced given the grid:

```
q)meta markout
c          | t f a
-----------| -----
tradeID    | j   s
grids      | F
markoutVal | F
matchedTime| P
```

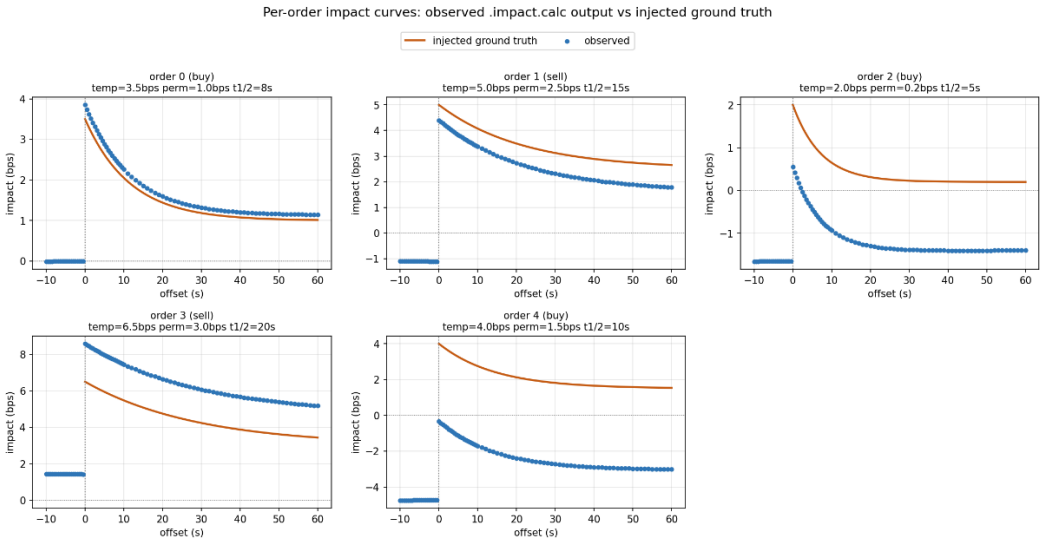
```
stale       | B
```

Real-time: same shape, same code pattern, twice

Both metrics need a live version, and both use the identical incremental pattern — a pending table of not-yet-reachable offsets, drained as new prices arrive:

```
.markout.onTrade / .markout.onRate
.impact.onOrder  / .impact.onBook
```

Structurally identical functions, different tables. That's not an accident of copying-and-pasting — it's the same underlying problem (accumulate partial results until enough time has passed to complete them) showing up twice because the shape of "value at T+Δ for many Δ" doesn't care whether T is a client fill or your own order.



Conclusions

Markout and market impact examine the same trade from two distinct perspectives: the former concerns the counterparty's experience, while the latter concerns one's own. Despite this difference in orientation, both are implemented according to the same underlying computational pattern. A grid is constructed once, the relevant rows are exploded against it, a single join is performed, and metric-specific logic is applied thereafter. Recognizing this shared structure transforms what would otherwise be two independent analytical implementations into a single, well-tested core supplemented by two comparatively thin, metric-specific layers.

Most of the initial effort is devoted to ensuring that this core is correct: attributes, column naming, and variable scoping, among other unglamorous details, must be handled properly. The remaining effort is devoted to ensuring that the metric-specific layer is rigorous rather than merely plausible. A fitted decay curve accompanied by a goodness-of-fit statistic is preferable to two values read from arbitrarily chosen windows, for the same reason that a sorted, attributed aj is preferable to an iterative loop: the former is not merely more efficient or more elegant, but genuinely verifiable.